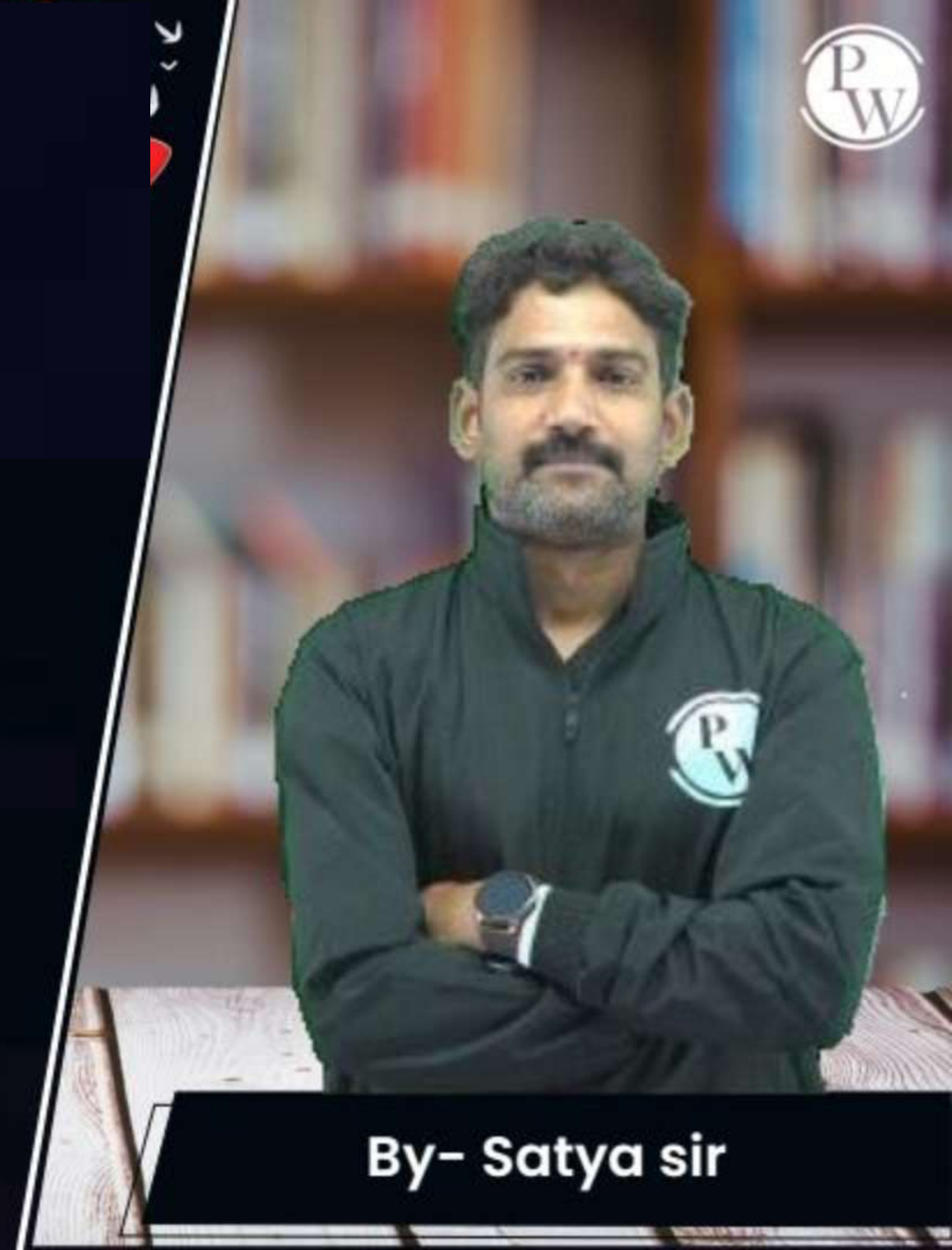
Computer Science & DA

Data Structure through Python

Linked lists

DPP - 01

Discussion Notes

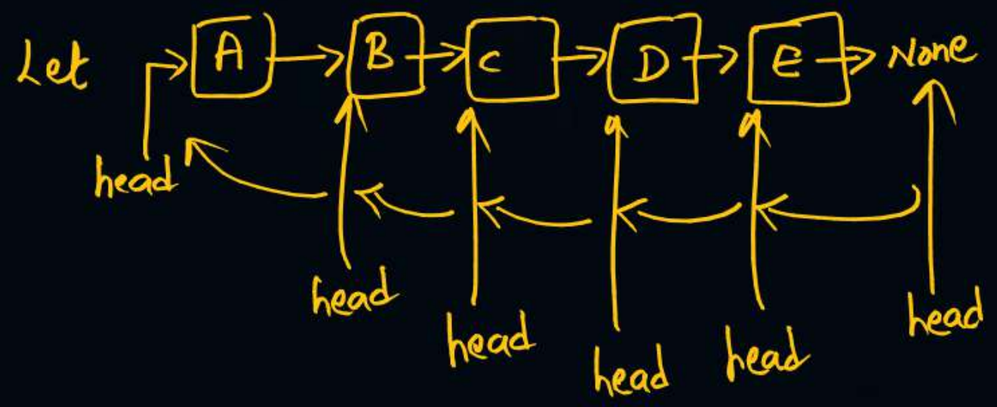


By- Satya sir

[MCQ]

#Q. What does the following function do for a given Linked List with first node as head?

```
def fun1(head):
    if head is None:
      return
    fun1(head.next)
    print(head.data, end=" ")
```



o/p: E D C B A

A. Prints all nodes of linked lists

B. Prints all nodes of linked list in reverse order

C. Prints alternate nodes of Linked List

D. Prints alternate nodes in reverse order

[MCQ]

#Q. The following function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution ?

```
class Node:
def __init__(self, value):
    self.value = value
    self.next = None
def rearrange(head):
if head is None or head.next is None:
    return
p = head
q = head.next
while q is not None:
# Swap values of p and q
temp = p.value.
p.value = q.value
q.value = temp
# Move p and q to next pair of nodes
p = q.next
if p is not None:
        q = p.next
else:
        q = None
# Helper function to print the linked list
def print_list(head):
        current = head
        while current is not None:
```

Empty list

Only 1 Node

(self.head)

o/p :- 2, 1, 4, 3, 6, 5, 7

```
  print(current.value, end=" ")
  current = current.next
print()
# Example usage:
if _name__ == "__main__":
    # Creating a linked list: 1–> 2–> 3–> 4–> 5
    head = Node(1)
    head.next = Node(2)
    head.next.next = Node(3)
    head.next.next.next = Node(4)
    head.next.next.next.next = Node(5)
    print("Original list:")
    print_list(head)
    rearrange(head)
    print("Rearranged list:")
    print_list(head)
```

A 1,2,3,4,5,6,7

B 2,1,4,3,6,5,7

C 1,3,2,5,4,7,6

D 2.3.4.5.6.7.1

[MCQ]

```
class Node:
def __init__(self, data)
    self.data = data
    self.Prev = None
    self.Next = None
```

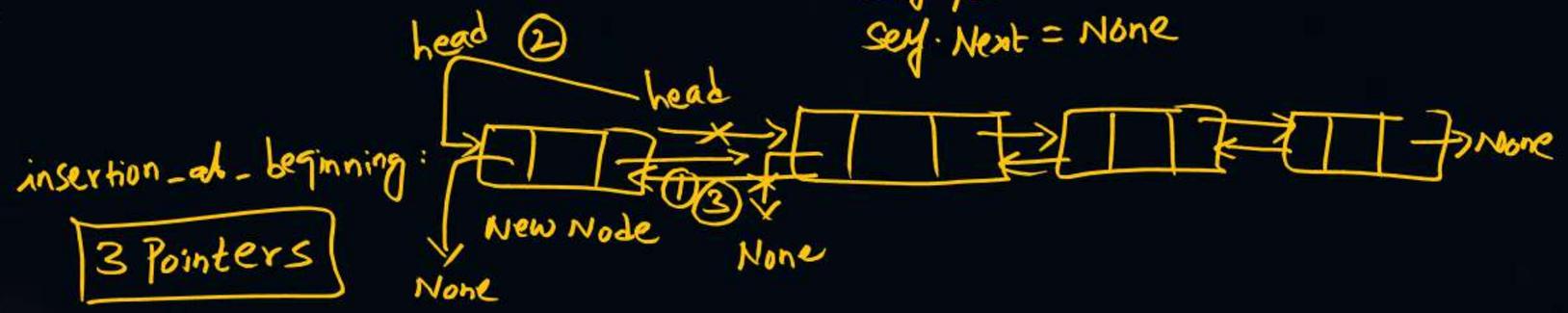
#Q. In a doubly linked list, the number of pointers affected for an insertion operation will be :

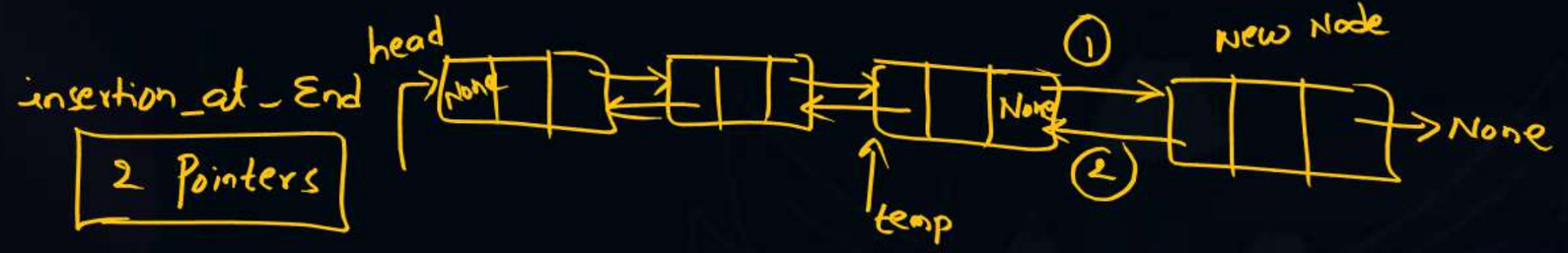
A. 5

B. 0

C. 1

D. None of these ✓





[MCQ]

#Q. Suppose each set is represented as a linked list with elements in arbitrary order. Which of the operations among union, intersection, membership, cardinality will be the slowest?

↳ Count : O(n)

≅ Searching O(n)

A. union

B. membership

C. cardinality

D. union, intersection ✓

L1: 1 → 2 → 3 → 4 → 5 → 6 (n nodes)

L2: 3 → 4 → 7 → 8 → 9 (m nodes) ⇒ m*n ⇒ O(n²)

L1∩L2 : 3 → 4

Union: L1∪L2 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 O(m+n) = O(n²) (or)

[MCQ]

#Q. Consider the function f defined below.

```
class Node:
def __init__(self, data):
    self.data = data
    self.next = None
def f(p):
    if p is None or p.next is None:
        return True
return p.data <= p.next.data and f(p.next)
```

Empty list only 1 Element

10 → 20 → 30 → 40 → None

p p.next

(Increasing order)

For a given linked list p, the function f returns 1 if and only if.

A. not all elements in the list have the same data value.

B. the elements in the list are sorted in non-decreasing (Increasing) order of data value

C. the elements in the list are sorted in non-increasing order of data value

D. None of them

[MCQ]

#Q. Which of the following points is/are true about Linked List data structure when it is compared with array ?

ANS: E

A. Arrays have better cache locality that can make them better in terms of performance.

Contiguous space

B. It is easy to insert and delete elements in Linked List [No requirement of shifting elements]

C. Random access is not allowed in a typical implementation of Linked Lists

D. The size of array has to be pre-decided, linked lists can change their size any time.

E. All of the above

[MCQ]

#Q. In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is.

All Nodes need to be compared | Traversed. = n Comparisons

A. log n

B. n/2

C. log n – 1

D. n ✓

[MCQ]

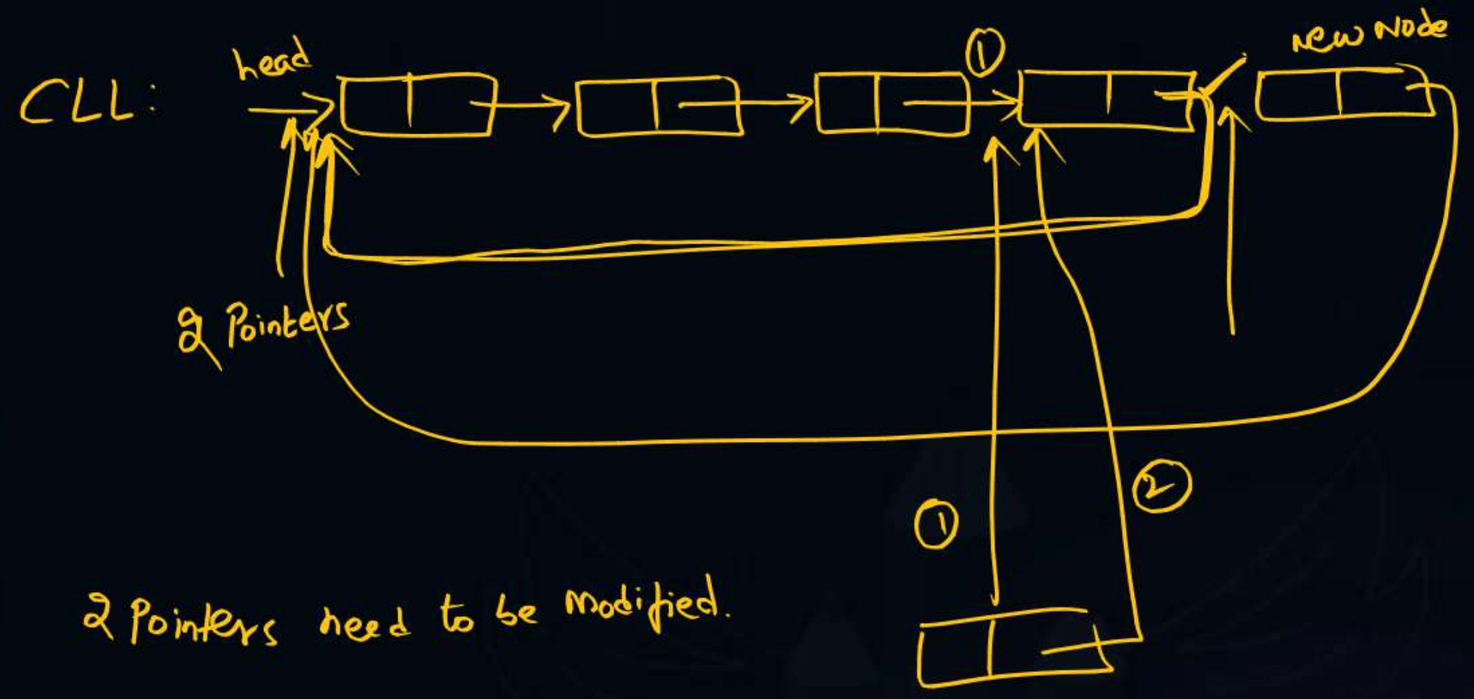
#Q. In a circular linked list organization, insertion of a record involves modification of

A. One pointer

B. Two pointers

C. Multiple pointers

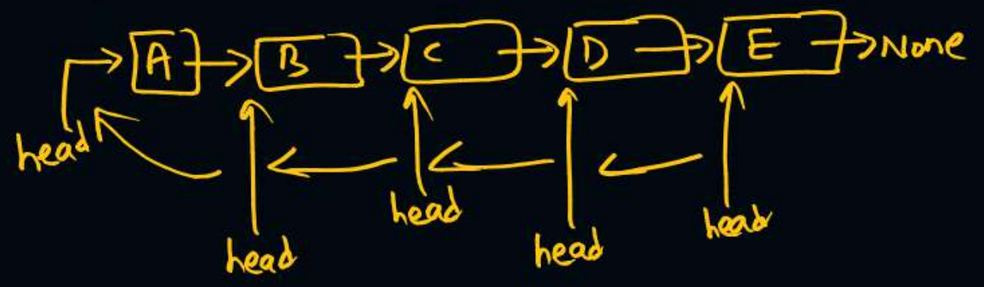
D. No pointer



[MCQ]

#Q. What does the following function do for a given Linked List with first node as head?

```
class Node:
def __init__(self, data):
    self.data = data
    self.next = None
def fun1(head):
if head.next is None:
    return
print(head.next.data, end=' ')
fun1(head.next)
```



o/p: B C D E

A. Prints all nodes of linked lists

B. Prints all nodes of linked list except first node ✓

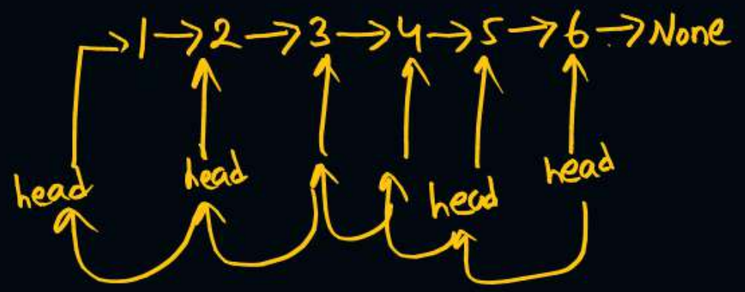
C. Prints alternate nodes of Linked List

D. Prints all nodes of linked list except last node

[MCQ]

#Q. What does the following function print for a given Linked List with input 1,2,3,4,5,6?

```
class Node:
def __init__(self, data):
    self.data = data
    self.next = None
def fun1(head):
    if head.next is None:
      return
print(head.data, end=' ')
fun1(head.next)
print(head.data, end=' ')
```



o/p: 1 2 3 4 5 5 4 3 2 1

A. 1,2,3,4,5,6,6,5,4,3,2,1

B. 1,2,3,4,4,3,2,1

C. 1,2,3,4,5,5,4,3,2,1 ✓

D. 1,2,3,4,5,1,2,3,4,5

THANK - YOU